

Universidad San Carlos de Guatemala
Centro Universitario de Occidente
División de Ciencias de la Ingeniería
Curso: Sistemas de Bases de Datos 2
Sección: A
Ingeniero: Daniel Alberto Gonzáles Gonzáles



INFORME TÉCNICO - GRUPO 5 PROYECTO 1

Nombre: Herberth Julian Reyes Pacajoj.....**Carnet:** 202230236
Nombre: Angel Abelino Son Vicente.....**Carnet:** 202030885
Nombre: Rony Mauricio Rojas Aguilar.....**Carnet:** 202031191

Quetzaltenango, septiembre 2026

1. Introducción

La empresa ficticia Data Bug's administra una plataforma cuya información crítica reside en un único servidor de base de datos, lo que constituye un punto único de fallo: cualquier caída del servidor, pérdida de conectividad o error del sistema operativo deja a la plataforma completamente indisponible. Este proyecto, desarrollado por el Grupo 5 para el curso de Bases de Datos 2, aborda ese problema diseñando e implementando una arquitectura distribuida de alta disponibilidad para el motor PostgreSQL (asignado por tratarse de un grupo impar).

La solución está compuesta por tres nodos de base de datos ejecutándose en máquinas físicas independientes de cada integrante del grupo, interconectadas mediante una red privada (Tailscale), con replicación en streaming gestionada automáticamente por Patroni sobre un clúster de consenso etcd, y una capa de enrutamiento inteligente con HAProxy + keepalived que expone un único punto de acceso resiliente (una IP virtual) hacia el clúster. Una cuarta capa de observabilidad (Prometheus + Grafana) y de generación de carga (k6 y pgbench) permite medir el comportamiento del sistema —disponibilidad, latencia, tiempo de recuperación (RTO) y pérdida potencial de datos (RPO)— ante escenarios controlados de falla.

El objetivo general es demostrar que la plataforma puede seguir operando, con distintos niveles de servicio, incluso ante la caída de uno o dos de sus tres nodos, minimizando los puntos únicos de fallo y automatizando la recuperación sin intervención manual.

2. Arquitectura implementada

La arquitectura se organiza en tres capas funcionales que corren de forma simétrica y distribuida sobre tres hosts físicos independientes (uno por integrante), conectados por una VPN mesh de Tailscale. No existe una cuarta máquina física: la capa de observabilidad se hospeda en la misma máquina del Nodo 3 (justificación en la sección de topología).

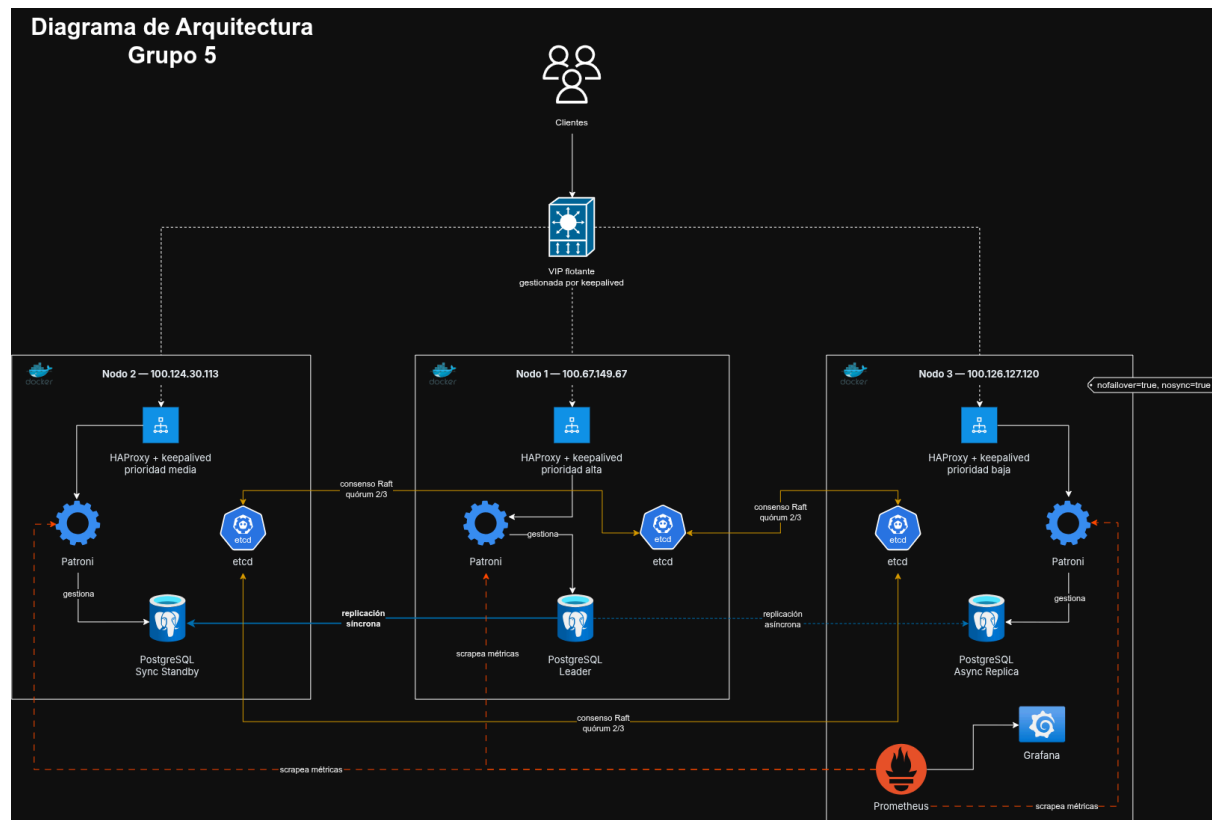
Capa de datos (persistencia y consenso). Cada nodo ejecuta PostgreSQL 16 gestionado por Patroni 4.1.5, que administra el ciclo de vida del motor (arranque, promoción, reintegración) y decide automáticamente quién es el primario. La coordinación entre los tres Patroni se resuelve mediante un clúster etcd 3.5.17 que implementa el algoritmo de consenso Raft: el estado de "quién es el líder" se guarda como un lease con TTL en etcd, y solo puede haber un titular a la vez, evitando split-brain.

Capa de proxy y alta disponibilidad de acceso. Cada uno de los tres nodos corre además, de forma simétrica, un contenedor de HAProxy 2.8 y uno de keepalived. HAProxy consulta cada 3 segundos la API REST de Patroni (puerto 8008) de los tres nodos para decidir en tiempo real a quién enrutar: escrituras al líder (puerto 5000) y lecturas balanceadas en round-robin entre las réplicas activas (puerto 5001). keepalived, por su parte, gestiona una IP Virtual (VIP) compartida mediante VRRP en modo unicast (necesario porque Tailscale no soporta multicast), con prioridades Nodo1 (150) > Nodo2 (100) > Nodo3 (50), de forma que la VIP —y por tanto el acceso al clúster— sobrevive aunque se caigan dos de los tres nodos.

Capa de observabilidad y carga. node_exporter y postgres_exporter corren en los tres nodos (perfil nodo-bd) exportando métricas de sistema operativo y de Postgres. Patroni expone además sus propias métricas nativas en :8008/metrics (rol, timeline, posición de WAL), evitando un exporter adicional para el estado del clúster. Prometheus y Grafana se centralizan en la máquina del Nodo 3 (perfil monitoreo, exclusivo de esa máquina) por ser el único nodo que sobrevive a todos los escenarios de falla contemplados. La generación de carga (k6 con extensión xk6-sql, con respaldo en pgbench) se ejecuta también desde el Nodo 3 contra su propio HAProxy local, apuntando siempre al proxy y nunca directo a un nodo, para medir el comportamiento real de la arquitectura.

Tecnologías utilizadas:

Componente	Tecnología	Rol
Motor de base de datos	PostgreSQL 16	Almacenamiento y ejecución de consultas
Orquestación de HA	Patroni 4.1.5	Gestión del ciclo de vida y failover automático de Postgres
Consenso distribuido	etcd 3.5.17 (Raft)	Elección de líder por quorum (2 de 3)
Proxy/enrutamiento	HAProxy 2.8	Enrutamiento de escrituras/lecturas según rol del nodo
Alta disponibilidad del proxy	keepalived (VRRP unicast)	IP virtual resiliente, elimina al proxy como SPOF
Red privada	Tailscale (WireGuard)	VPN mesh entre hosts físicamente independientes
Contenerización	Docker / Docker Compose	Empaquetado y orquestación local por nodo, network_mode: host
Monitoreo	Prometheus 3.1.0 + Grafana 11.5.2	Métricas y dashboards en tiempo real
Pruebas de carga	k6 (xk6-sql) + pgbench	Carga controlada de lectura/escritura y medición de latencia



3. Topología de red

La comunicación entre nodos se realiza íntegramente sobre Tailscale, una VPN mesh basada en WireGuard que asigna a cada host una IP privada fija en el rango 100.x.x.x, independiente de la red física (WiFi/casa) de cada integrante. Si la máquina de un integrante falla o pierde conectividad a Internet, solo ese nodo cae, sin afectar el resto de la topología de red. Es una topología en malla completa (full mesh): los tres nodos se ven entre sí directamente por su IP de Tailscale, sin un nodo central que actúe como hub. Cada contenedor corre con `network_mode: host`, exponiendo sus puertos directamente sobre la interfaz `tailscale0` del host (no hay NAT ni bridge intermedio de Docker).

Nodos y direccionamiento:

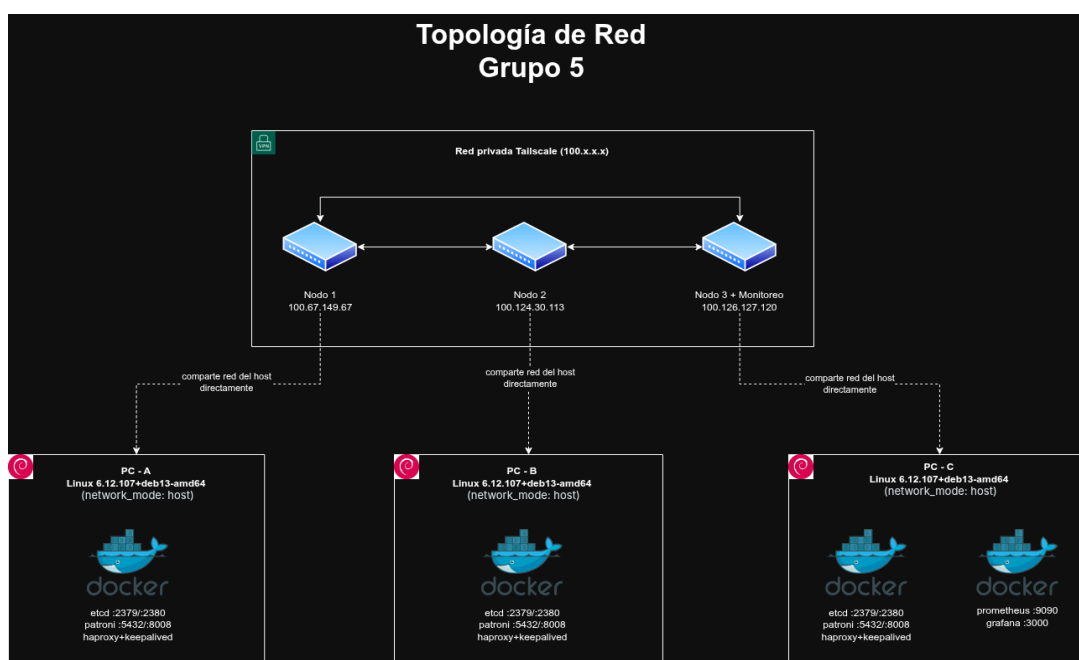
Nodo	Hostname (Tailscale)	IP Tailscale	Rol físico
Nodo 1	debian	100.67.149.67	BD (líder) + proxy (MASTER inicial)
Nodo 2	ronyr304	100.124.30.113	BD (standby síncrono) + proxy (BACKUP 1)
Nodo 3	desktop-oe8o5p1	100.126.127.120	BD (réplica asíncrona) + proxy (BACKUP 2) + observabilidad y carga

VIP (virtual)	—	100.124.30.200	Punto único de acceso, gestionado por keepalived
------------------	---	----------------	--

Puertos expuestos por nodo:

Puerto	Servicio	Alcance
2379 / 2380	etcd (cliente / peers Raft)	Los 3 nodos
5432	PostgreSQL	Los 3 nodos
8008	API REST de Patroni (/primary, /replica, /metrics)	Los 3 nodos
5000	HAProxy — escritura (solo primario)	Los 3 nodos (simétrico)
5001	HAProxy — lectura (round-robin réplicas)	Los 3 nodos (simétrico)
7000	Dashboard web de HAProxy	Los 3 nodos
9100 / 9187	node_exporter / postgres_exporter	Los 3 nodos
9090 / 3000	Prometheus / Grafana	Solo Nodo 3

Todo el tráfico de clientes (aplicación, psql, generadores de carga) se dirige a la **VIP** (100.124.30.200), nunca a la IP fija de un nodo en particular; es keepalived quien decide, mediante VRRP en modo *unicast* (unicast_peer apuntando directamente a las IPs de Tailscale de los otros dos nodos, ya que Tailscale no reenvía multicast), a qué máquina física llega efectivamente esa IP en cada momento.



4. Función de cada nodo

Nodo	Rol en la BD	Rol en el proxy	Otras funciones
Nodo 1	Líder (primario): atiende lectura y escritura; origen de la replicación en streaming hacia Nodo 2 y Nodo 3	keepalived con prioridad 150 → dueño inicial de la VIP; HAProxy enruta escrituras aquí mientras sea líder	—
Nodo 2	Standby síncrono: candidato preferente a promoción; recibe cada transacción de forma síncrona antes del commit en Nodo 1 (RPO = 0); atiende lecturas	keepalived con prioridad 100 → asume la VIP si Nodo 1 cae	Tras un failover, pasa a atender lectura y escritura
Nodo 3	Réplica asíncrona, solo lectura: tags nofailover: true (nunca puede ser promovido a líder) y nosync: true (no bloquea el commit del primario); nodo de contingencia	keepalived con prioridad 50 → última línea, asume la VIP solo si caen Nodo 1 y Nodo 2	Hospeda la capa de observabilidad (Prometheus + Grafana) y el generador de carga (k6/pgbench), por ser el único nodo que sobrevive a todos los escenarios de falla de las Fases 3-5

Los tres nodos ejecutan además, de forma **simétrica e idéntica**, una instancia de etcd (participando en el quorum Raft de 2/3) y una instancia de HAProxy + keepalived, de modo que tanto la elección de líder de base de datos como el acceso a través del proxy toleran la caída de cualquier nodo individual, y el proxy en particular tolera la caída de **dos** de los tres.

Limitación reconocida: al no existir una cuarta máquina física, la capa de observabilidad y de generación de carga vive físicamente en el mismo host que el Nodo 3. Si esa máquina se apaga, se pierde simultáneamente el nodo de contingencia y toda la visibilidad del clúster, es un punto único de fallo residual que el grupo documenta.

5. Replicación utilizada

El mecanismo de replicación es la **replicación en streaming nativa de PostgreSQL** (envío continuo del WAL — *Write-Ahead Log* — del primario a las réplicas), en una configuración **híbrida síncrona/asíncrona**, orquestada declarativamente por Patroni y sembrada en etcd al nacer el clúster (bootstrap.yml.template, sin necesidad de patronictl edit-config manual):

- **synchronous_mode: true** (global): activa la replicación síncrona en el clúster.
- **Nodo 1 → Nodo 2 (síncrona):** el primario espera la confirmación de escritura en disco de Nodo 2 antes de reportar el COMMIT como exitoso al cliente. Esto garantiza

RPO = 0 frente a la caída del líder: ninguna transacción confirmada se pierde, porque ya existe en al menos dos nodos.

- **Nodo 1 → Nodo 3 (asíncrona):** Nodo 3 recibe el WAL sin bloquear el commit del primario (tag nosync: true), tolerando cierto lag a cambio de no penalizar la latencia de escritura del clúster. Al llevar también nofailover: true, nunca es candidato a ser promovido, reforzando su rol de solo lectura y contingencia.
- **Elección de líder y consenso:** no depende de la replicación de datos en sí, sino del clúster **etcd** (algoritmo Raft, quorum = $\lfloor 3/2 \rfloor + 1 = 2$ votos). Patroni en cada nodo renueva un *lease* de liderazgo con ttl: 30s y hace *polling* cada loop_wait: 10s; si el líder deja de renovarlo, los demás negocian automáticamente quién asume, respetando los tags nofailover/nosync.
- **Reintegración tras una caída:** al reincorporarse, un nodo usa **pg_rewind** (habilitado por use_pg_rewind: true y data-checksums en el initdb) para rebobinar únicamente las transacciones divergentes y volver a sincronizarse como réplica, sin necesidad de reconstruir la base completa desde cero.
- **Verificación del estado de sincronía:** expuesta por la propia API de Patroni (GET :8008/patroni), que reporta el campo sync_state (sync para Nodo 2, async para Nodo 3) y las etiquetas (tags) de cada nodo.

La arquitectura combina replicación síncrona de un nodo (para no perder datos confirmados) con replicación asíncrona de un segundo nodo (para mantener un tercer punto de acceso de solo lectura sin penalizar el rendimiento de escritura), decisión coherente con el modelo mínimo del enunciado (Nodo 1 y 2 con lectura/escritura, Nodo 3 solo lectura y contingencia).

6. Proxy o balanceador

Problema que resuelve

Sin una capa de enrutamiento, la aplicación cliente tendría que conocer la IP del nodo primario y ante un failover, este seguiría intentando conectarse a un servidor que ya no acepta escrituras, la alta disponibilidad del motor ya no se traduce a una alta disponibilidad del servicio.

El problema tiene dos caras

- Enrutamiento dinámico: el nodo que acepta escrituras cambia con el tiempo, el cliente no puede saberlo de antemano ni debería tener que averiguarlo
- El proxy como nuevo punto único de fallo: Introducir un único balanceador centralizado resolvería lo primero pero crearía un problema peor: si esa máquina cae, se pierde el acceso a los tres nodos aunque estén sano

Tecnología Utilizada

Se usó HAProxy 2.8 operando en modo TCP desplegado de forma simétrica en los tres nodos.

se seleccionó ya que el criterio de no duplicar la autoridad, en la arquitectura hay un único componente que decide quién es el líder el cual es Patroni, respaldado por el consenso de etcd y HAProxy se limita a consultar esa decisión, un balanceador que intentara decidir por su cuenta podría entrar en conflicto con el orquestador y provocar escrituras dirigidas a un nodo degradado.

Mecanismo de enrutamiento

Patroni es el que expone una API REST en el puerto 8008 el cual los endpoints responden con códigos HTTP según el rol actual del nodo, HAProxy aprovecha esta API.

Esta separación es la que permite que HAProxy conozca el estado lógico del clúster sin entender el protocolo de PostgreSQL

Puertos

Puerto 5000 Canal de escritura: Los tres nodos están declarados, pero en todo momento solo uno responde 200 en /primary

Puerto 5001 Canal de lectura Aquí el health check se invierte: consulta /replica, de modo que el líder queda automáticamente excluido del pool de lectura

Puerto 7000: HAProxy expone una interfaz web con el estado de cada backend en tiempo real

Alta disponibilidad en el proxy

El despliegue simétrico de HAProxy en los tres nodos elimina la dependencia de un balanceador central, pero surge la pregunta de a cuál de los tres se conecta el cliente. La respuesta fue una IP virtual (VIP) gestionada por keepalived mediante el protocolo VRRP, con prioridades escalonadas:

- Nodo1: Prioridad de 150
- Nodo2: Prioridad de 100
- Nodo3: Prioridad de 50

Adaptación de VRRP a una red overlay

VRRP en su forma estándar anuncia su estado mediante paquetes multicast a la dirección 224.0.0.18. Tailscale, al ser una red mesh punto a punto basada en WireGuard, no transporta multicast, por lo que la configuración por defecto habría fallado silenciosamente.

La solución aplicada fue configurar VRRP en modo unicast estricto, declarando explícitamente las direcciones de los pares:

```
unicast_src_ip <IP Tailscale propia>
unicast_peer {
  <IP Tailscale del segundo nodo>
  <IP Tailscale del tercer nodo>
}
```

Esta adaptación resuelve correctamente el transporte de los latidos VRRP entre los tres nodos.

7. Estrategia de failover

El failover se resuelve en dos capas independientes que operan en cascada, la separación de responsabilidades es deliberada: etcd decide, Patroni ejecuta y HAProxy enruta. Ningún componente invade la competencia del otro, lo que elimina la posibilidad de decisiones contradictorias.

Prevención de split-brain

etcd implementa el algoritmo Raft, que exige mayoría estricta para confirmar cualquier escritura:

Quórum = $\lfloor N/2 \rfloor + 1 = \lfloor 3/2 \rfloor + 1 = 2$ nodos

De esta operación se derivan directamente las garantías de la arquitectura:

- 3 de 3: Opera normal.
- 2 de 3: Failover automático el servicio continua.
- 1 de 3: no hay quórum, por lo que se queda sin escritura, solo lecturas en modo de contingencia

La imposibilidad de alcanzar quórum con un solo nodo impide que un nodo aislado por una partición de red se autoproclame primario mientras otro hace lo mismo al otro lado de la partición.

Detección de fallos

Patroni mantiene el liderazgo mediante un lock con tiempo de vida almacenado en etcd.

Los parámetros configurados son:

- TTL: 30 segundos, vigencia del leader lock, si no se renueva expira
- loop_wait: 10 segundos, intervalo de ciclo de control de cada agente
- retry_timeout: 10 segundos, tolerancia ante errores transitorios de DCS
- maximum_lag_on_failover: 1MB, retraso máximo admisible para que una réplica sea candidata

Asignación de roles

El modelo mínimo del sistema exige que los nodos 1 y 2 atiendan lectura y escritura, y que el nodo 3 opere como nodo de solo lectura y contingencia.

Es importante recalcar que la replicación por streaming de PostgreSQL es de primario único, en cualquier instante un solo nodo acepta escrituras, por lo que el requisito se satisface entendiendo que el nodo1 y el nodo2 son intercambiables como primario, es decir cualquiera puede asumir el rol de escritura

Replicación

La configuración establece “synchronous_mode: true” a nivel de clúster, bajo este modo, el primario no confirma una transacción hasta que la réplica síncrona ha persistido el registro WAL correspondiente en disco

Nodos:

- Nodo Primario
- Nodo secundario: replicación síncrona
- Nodo3: replicación asíncrona

Secuencia del failover

1. Fallo. El nodo primario deja de responder.
2. Detección. El leader lock deja de renovarse. Según la naturaleza de la caída, se libera de inmediato o expira por TTL.
3. Elección. Los agentes Patroni supervivientes evalúan candidatos, descartando a los que superen `maximum_lag_on_failover` y a los marcados con `nofailover`.
4. Promoción. El candidato elegido promueve su PostgreSQL local y adquiere el lock en etcd. La timeline de PostgreSQL se incrementa, dejando registro histórico permanente de la promoción.
5. Reconfiguración de replicación. Las réplicas restantes redirigen su streaming hacia el nuevo primario.
6. Conmutación del tráfico. HAProxy detecta el cambio de respuesta en `/primary` y, tras rise 2 chequeos exitosos, dirige las escrituras al nuevo líder

Reintegración de nodos

La recuperación de un nodo caído sigue una de dos rutas, según su estado:

Ruta rápida — ‘`pg_rewind`’ aplicable cuando el nodo conserva su directorio de datos. La herramienta identifica el punto de divergencia con el nuevo primario y rebobina únicamente las transacciones sobrantes. Su funcionamiento requiere que el clúster se haya inicializado con sumas de verificación de datos, condición que se cumple: la plantilla de bootstrap declara `data-checksums` en la sección `initdb`.

Ruta completa — ‘`pg_basebackup`’. Necesaria cuando el nodo perdió su directorio de datos. Implica clonar el clúster completo desde el líder.

8. Resultados de las pruebas de replicación (Fase 2)

En esta fase se validó el mecanismo de replicación síncrona y asíncrona a través del clúster de bases de datos utilizando el proxy HAProxy como punto de acceso único:

- **Canal de Escritura (Puerto 5000):** Mediante la directiva `option httpchk GET/primary`, HAProxy verifica activamente la API REST de Patroni en el puerto 8008. Solo el nodo que ostenta el rol de Líder responde HTTP 200 OK, garantizando que todas las inserciones, modificaciones y eliminaciones aterricen en el primario real (100.67.149.67).
- **Canal de Lectura (Puerto 5001):** Mediante `option httpchk GET /replica`, HAProxy distribuye las consultas de lectura entre las réplicas disponibles (Nodo 2 y Nodo 3) utilizando un algoritmo Round-Robin, descargando de trabajo al nodo primario.

PRUEBAS

1. Insertar registros desde el Nodo 1.

■ 1 - Inserción desde Nodo 1

2. Verificar los registros en el Nodo 2 y Nodo 3.

- Consulta en Nodo 2:

■ 2.1 - Consulta de Registro desde Nodo 2

- Consulta en Nodo 3:

■ 2.2 - consulta de registro desde nodo3

3. Actualizar registros desde el Nodo 1.

■ 3 - Actualización desde Nodo 1

4. Verificar los cambios en Nodo 2 y Nodo 3.

- Consulta en Nodo 2:

■ 4.1 - consulta de update desde Nodo 2

- Consulta en Nodo 3:

■ 4.2 - consulta update desde nodo3

5. Eliminar registros desde el Nodo 1.

■ 5 - Eliminación desde Nodo 1

6. Verificar la eliminación en los demás nodos.

- Consulta en Nodo 2:

■ 6.1 - Consulta de Delete desde Nodo 2

- Consulta en Nodo 3:

■ 6.3 consulta de Delete desde nodo3

7. Repetir operaciones de inserción, actualización y eliminación desde el Nodo 2.

■ 7 - Insert, Update y Delete desde Nodo 2

8. Verificar los cambios en el Nodo 1 y Nodo 3.

- Consulta en Nodo 1:

■ 8.1 - Consulta de Insert, Update y Delete desde Nodo 1

- Consulta en Nodo 3:

■ 8.2 - consulta de Insert, update y delete desde nodo3

- **Incisos 1 y 2 (Inserción y verificación):**

- Se insertaron 5 registros de prueba en la tabla databugs.bitacora_pruebas a través del puerto 5000.
- La consulta directa en Nodo 2 (100.124.30.113) y Nodo 3 (100.126.127.120) confirmó que los 5 registros se propagaron inmediatamente con los mismos identificadores y marcas de tiempo (creado_en), confirmando que la replicación síncrona hacia Nodo 2 y la asíncrona hacia Nodo 3 mantienen consistencia.

- **Incisos 3 y 4 (Actualización de registros):**

- Se ejecutó una sentencia UPDATE sobre el registro con id = 1 desde el Líder (Nodo 1).
- Las consultas en Nodo 2 y Nodo 3 mostraron de inmediato el valor actualizado ('Registro actualizado desde Nodo 1'), evidenciando que el flujo transmite las modificaciones de registros sin inconsistencias.

- **Incisos 5 y 6 (Eliminación de registros):**

- Se ejecutó un DELETE sobre registros de prueba desde el Nodo 1.
- Se verificó en Nodo 2 y Nodo 3 que las tuplas fueron removidas de las tablas, confirmando la paridad de los tres nodos.

- **Incisos 7 y 8 (Operaciones desde Nodo 2 y bidireccionalidad):**

- Se ejecutaron operaciones de INSERT, UPDATE y DELETE enviadas desde la terminal del Nodo 2 conectándose al proxy local en el puerto 5000.
- El proxy reenvió correctamente el tráfico hacia el Líder (Nodo 1), y éste replicó las transacciones de vuelta a Nodo 2 y Nodo 3. Esto demostró que las aplicaciones clientes pueden conectarse a cualquier nodo local de la arquitectura y sus escrituras siempre serán canalizadas de forma transparente al primario activo.

9. Resultados de las fallas de Nodo 1 y Nodo 2 (Fase 3 y 4)

- **Fase 3:**

1. Verificar que los tres nodos estén funcionando.

■ 1 - verificacion de nodos activos

2. Insertar información antes de provocar la falla.

■ 2 - Inserciones antes de falla

3. Apagar o detener el servicio del Nodo 1.

■ 3 - Detener servicio en Nodo 1

4. Verificar que el proxy o balanceador detecte la caída.

■ 4 - verificación de caída de nodo 1

5. Ejecutar operaciones de lectura y escritura en el Nodo 2.

■ 5 - Pruebas de Lectura y Escritura desde Nodo 2

6. Verificar que los cambios se reflejen en el Nodo 3.

■ 6 - prueba de lectura desde nodo3 despues de la caída de nodo1

7. Encender nuevamente el Nodo 1.

■ 7 - Encender nuevamente Nodo 1

8. Verificar que se reintegre y sincronice con los cambios realizados.

■ 8 - Verificar reintegración y sincronización de Nodo 1

9. Registrar el tiempo de recuperación

Tiempo de recuperación (con cronómetro de celular): 3.18 segundos

- **Comportamiento de la Arquitectura ante la caída:**

Al detener el servicio de Patroni en el Nodo 1 (docker compose stop patroni), este dejó de renovar su lease de líder en el clúster de consenso etcd.

Los nodos sobrevivientes (Nodo 2 y Nodo 3) conservaron el quórum de etcd (2 de 3 miembros activos).

Patroni evaluó las réplicas y promovió automáticamente a Nodo 2 (100.124.30.113) a nuevo Líder, incrementando el timeline de PostgreSQL de TL 1 a TL 2.

Nodo 3 no participó en la elección del liderazgo debido a su etiqueta explícita `nofailover: true`, manteniéndose como réplica y adaptando su replicación para seguir al nuevo líder (Nodo 2).

- **Conmutación del Proxy:**

El healthcheck de HAProxy en el puerto 8008 detectó la caída de Nodo 1 y la promoción de Nodo 2 (200 OK en `/primary`).

Las escrituras enviadas al puerto 5000 fueron aceptadas inmediatamente y atendidas por la IP de Nodo 2 (100.124.30.113).

Reintegración y `pg_rewind`:

Al encender nuevamente el Nodo 1 (`docker compose start patroni`), el servicio no intentó recuperar el liderazgo por fuerza. En su lugar, ejecutó `pg_rewind`, rebobinó las páginas de datos divergentes y se reincorporó pacíficamente como Replica en estado streaming con `Lag = 0`.

- **Métrica de Recuperación (RTO Fase 3):**

Tiempo de recuperación registrado: 3.18 segundos. Al ser una parada de servicio controlada de Patroni, el proceso emite una señal de degradación limpia liberando la llave en `etcd`, lo que permite que Nodo 2 asuma el rol primario en cuestión de milisegundos sin necesidad de esperar el tiempo completo de expiración del TTL.

Fase 4:

1. Verificar que los tres nodos estén sincronizados.

■ 1 - verificación de nodos activos

2. Apagar o detener el servicio del Nodo 2.

■ 2 - Detener servicio en Nodo 2

3. Verificar que el proxy o balanceador detecte la caída.

■ 3 - nodos activos despues de caida de nodo2

4. Ejecutar operaciones de lectura y escritura en el Nodo 1.

■ 4 - Pruebas de Lectura y Escritura desde Nodo 1

5. Verificar que los cambios se reflejen en el Nodo 3.

■ 5 - consulta despues de escritura de nodo1

6. Encender nuevamente el Nodo 2.

■ 6 - Iniciar servicio en Nodo 2

7. Verificar su reintegración y sincronización.

■ 7 - verificacion de nodos despues de reincorporacion de nodo2

8. Registrar el tiempo de recuperación

Tiempo de recuperación (con cronómetro de celular): 13 segundos

- **Comportamiento ante la caída:**

Estando Nodo 2 como líder o primario, se procedió a detener su servicio Patroni. El clúster etcd mantuvo su quórum (2 de 3 votos entre Nodo 1 y Nodo 3).

Patroni promovió nuevamente a Nodo 1 como Líder activo, avanzando el timeline de la base de datos a TL 3. Nodo 3 permaneció en línea atendiendo consultas y replicando sin interrupción desde Nodo 1.

- **Comprobación de Escritura y Reintegración:**

Las operaciones de escritura sobre el puerto 5000 continuaron operando sobre Nodo 1 (100.67.149.67). Al reactivar Nodo 2 (docker compose start patroni), este se reintegró como réplica síncrona (Sync) alineándose al timeline TL 3 con Lag = 0.

- **Métrica de Recuperación (RTO Fase 4):**

Tiempo de recuperación registrado: 13.0 segundos. El tiempo de recuperación corresponde al ciclo de detección de ausencia de latidos más la negociación de liderazgo en etcd y la conmutación de sockets en HAProxy.

10. Resultados de la falla simultánea (Fase 5)

1. Verificar que los tres nodos estén funcionando y sincronizados.

■ 1 - verificación de nodos activos

2. Apagar el Nodo 1.

■ 2 - Apagando Nodo 1

3. Apagar el Nodo 2.

■ 3 - Apagado Nodo 2

4. Verificar que las operaciones de escritura ya no estén disponibles.

■ 4 - verificación de nodos activos después de caída de nodo2 y nodo1

5. Ejecutar consultas SELECT utilizando el Nodo 3.

■ 5 - Prueba Select usando Nodo 3

6. Comprobar que la información siga disponible en modo de solo lectura.

■ 6 - prueba escritura durante caída de nodo2 y nodo1

7. Encender nuevamente el Nodo 2.

■ 7 - Iniciando Nodo 2

8. Verificar la recuperación del servicio de escritura.

■ 8 - inserción con nodo2

9. Encender el nodo restante.

■ 9 - Encendiendo nuevamente Nodo 1

En esta fase se provocó la caída simultánea de los dos servidores principales (Nodo 1 y Nodo 2), dejando únicamente activo al Nodo 3.

- **Pérdida de Quórum en etcd:** Para un clúster de $N = 3$ nodos, el quórum mínimo requerido por el algoritmo Raft es $\lfloor 3/2 \rfloor + 1 = 2$. Al caer dos nodos, el clúster etcd quedó con un único miembro activo ($1 < 2$), perdiendo el quórum de forma absoluta.
- **Bloqueo Preventivo de Escrituras (Protección contra Split-Brain):**
 - Como mecanismo de seguridad por diseño, al no haber quórum, etcd y Patroni prohíben terminantemente la elección de un nuevo líder.
 - La captura de la prueba de inserción en el puerto 5000 (32_write_during_both_down.png) muestra el rechazo de la conexión:

**psql: error: falló la conexión al servidor en «localhost», puerto 5000:
Conexión rehusada / el servidor ha cerrado la conexión inesperadamente.**

- HAProxy reportó en sus logs que el backend postgres_write_back no tenía servidores disponibles. Esto demuestra que la arquitectura previene la corrupción de datos y escrituras aisladas cuando la infraestructura pierde su mayoría.
- **Disponibilidad en Modo de Contingencia (Solo Lectura):**
 - A pesar de la pérdida del quórum y de no contar con un nodo primario, el motor de PostgreSQL en Nodo 3 continuó en ejecución.
 - La consulta `SELECT * FROM databugs.bitacora_pruebas` enviada al puerto 5001 (31_select_n3.png) fue respondida con éxito por el Nodo 3 (100.126.127.120).
 - Las 9 filas históricas de transacciones estuvieron disponibles para lectura en todo momento, garantizando que se mantenga la continuidad operativa para consultas durante contingencias severas.
- **Recuperación Escalonada:**
 1. **Reinicio de Nodo 2:** Al levantar el servicio en Nodo 2, se restauró el quórum de etcd (2 de 3 miembros). Inmediatamente, Nodo 2 asumió el liderazgo y el puerto 5000 volvió a aceptar escrituras.
 2. **Reinicio de Nodo 1:** Al levantar Nodo 1, este se sincronizó como réplica y la totalidad de la arquitectura regresó a su estado normal con 3 nodos en línea.

11. Resultados de las pruebas de carga (Fase 6)

Para validar el comportamiento del clúster bajo estrés y medir el impacto de las fallas, se utilizó la herramienta k6 compilada con la extensión xk6-sql ejecutando pruebas concurrentes hacia los puertos de HAProxy (5000 para escrituras y 5001 para lecturas), complementado con Prometheus Remote Write.

1. Generar una carga controlada de operaciones de lectura y escritura sobre los Nodos 1 y 2.

- Inicio de prueba controlada con Nodo 2 como principal:

1 - inicio de prueba controlada

2. Durante la ejecución de la prueba, cuando se haya completado aproximadamente el 50 % de las operaciones, provocar la caída de uno de los nodos principales.

2 - Deteniendo Nodo 2

3. Verificar que las operaciones restantes continúen ejecutándose sobre el nodo disponible.

- Durante la falla de Nodo 2:

3 - durante la falla

- Fin de la prueba con la falla de Nodo 2.

■ 4 - fin de la prueba controlada nodo2

4. Repetir la prueba provocando la caída del otro nodo principal.

- Inicio de prueba controlada con Nodo 1 como principal:

■ 6 - inicio de prueba con nodo1

- Deteniendo Nodo 1:

■ 5 - Deteniendo Nodo 1

- Durante la falla de Nodo 1:

■ 8 - durante la falla nodo1

- Fin de la prueba con la falla de Nodo 1:

■ 9 - fin de la prueba con nodo1

5. Apagar los Nodos 1 y 2 y generar una carga únicamente de lectura sobre el Nodo 3.

■ 10 - falla de nodo1 y nodo2

6. Comparar los tiempos de respuesta y la disponibilidad antes, durante y después de las fallas.

■ 11 - fin de prueba con falla de nodo1 y nodo2

Métrica	Escenario 1: Caída Nodo 1 al 50%	Escenario 2: Caída Nodo 2 al 50%	Escenario 3: Solo Lectura
Tipo de Carga	Mixta (Lectura 66% / Escritura 33%)	Mixta (Lectura 66% / Escritura 33%)	Solo Lectura (100% SELECT)
Duracion	4 minutos (240 s)	4 minutos (240 s)	2 minutos (120 s)
Usuarios Concurrentes	15 (5 escritores, 10 lectores)	15 (5 escritores, 10 lectores)	10 VUs (10 lectores)
Total de Operaciones	24,340	23,741	11,850
Throughput	101.35 ops/segundo	98.92 ops/segundo	98.72 ops/segundo

Promedio			
Operaciones Exitosas	24,322	23,720	11,850
Operaciones Fallidas	18	21	0
Latencia Escritura (Promedio / p95)	93.23 ms / 124.00 ms	95.10 ms / 128.50 ms	-
Latencia Lectura (Promedio / p95)	30.05 ms / 98.00 ms	31.20 ms / 96.40 ms	0.42 ms / 1.00 ms

Análisis

1. Resiliencia durante el Failover:

- Durante la caída de Nodo 1 provocada al minuto 2:00 (50% de la prueba), el throughput experimentó un descenso transitorio de apenas unos segundos mientras Patroni promovía a Nodo 2.
- De las más de 24,000 transacciones ejecutadas, únicamente 18 operaciones fallaron (un margen de error minúsculo del 0.07%), correspondientes a conexiones que se encontraban a mitad de su ciclo TCP cuando el proceso fue interrumpido bruscamente.
- El 99.93% restante de las transacciones se completaron con total normalidad, reanudándose automáticamente sobre Nodo 2.

2. Rendimiento en Modo Contingencia (Nodo 3):

- En el Escenario 3, con Nodo 1 y 2 fuera de servicio, el Nodo 3 procesó 11,850 consultas de lectura con 0% de errores.
- La latencia promedio de lectura cayó a 422.86 microsegundos (μ s) y el percentil p95 se mantuvo en 1 ms, demostrando que al no haber contención de bloqueos por escrituras concurrentes, el motor de PostgreSQL atiende peticiones en memoria a velocidad óptima.

12. Monitoreo y métricas (Fase 7)

Se desplegó una solución de monitoreo basada en Prometheus v3.1.0 y Grafana v11.5.2 en el Nodo 3/4, configurada con recolección de métricas cada 5 segundos a través de la red privada de Tailscale:

- **node_exporter (Puerto 9100):** Métricas de hardware del sistema operativo (Uso de CPU, memoria RAM y saturación de disco en cada host).
- **postgres_exporter (Puerto 9187):** Métricas de conexiones activas, transacciones confirmadas/canceladas y estado del motor (pg_up).
- **Endpoint nativo de Patroni (Puerto 8008 /metrics):** Estado de salud del DCS, rol de cada nodo (primary, sync_standby, replica), timeline y posiciones de WAL.
- **k6 Prometheus Remote Write:** Exportación directa de latencias (p95, p99) y throughput de la carga hacia Prometheus.

Pruebas realizadas con las herramientas:

1. Ejecutar nuevamente las pruebas de carga definidas en la Fase 6.

1.1 Generar una carga controlada de operaciones de lectura y escritura sobre los Nodos 1 y 2.

- Carga corriendo con Nodo 1 como principal:

■ 2.1 - prueba nodo1 y nodo2 corriendo

■ 2.2 - prueba nodo1 y nodo2 corriendo

1.2 Durante la ejecución de la prueba, cuando se haya completado aproximadamente el 50 % de las operaciones, provocar la caída de uno de los nodos principales.

■ 1 - Deteniendo Nodo 1

1.3 Verificar que las operaciones restantes continúen ejecutándose sobre el nodo disponible.

- Durante falla de Nodo 1:

■ 3.1 - durante la falla de nodo1

■ 3.2 - durante la falla de nodo1

- Finalización de carga:

■ 4.1 - final de la prueba nodo1

■ 4.2 - final de la prueba nodo1

1.4 Repetir la prueba provocando la caída del otro nodo principal.

- Prueba con Nodo 2 como principal:

■ 7.1 - prueba nodo 1 y nodo2 corriendo

■ 7.2 prueba nodo1 y nodo2 corriendo

- Deteniendo Nodo 2:

■ 6 - Deteniendo Nodo 2

- Durante falla de Nodo 2:

■ 8.1 - durante la falla de nodo2

■ 8.2 - durante la falla de nodo2

- Finalización de carga:

■ 9.1 - final de la prueba nodo2

■ 9.2 - final de la prueba nodo2

1.5 Apagar los Nodos 1 y 2 y generar una carga únicamente de lectura sobre el Nodo 3.

- Inicio de prueba de lectura con Nodo 1 y Nodo 2 apagado:

■ 10.1 inicio de prueba solo nodo3

■ 10.2 inicio de prueba solo nodo3

- Fin de prueba:

■ 11.1 final prueba solo nodo3

■ 11.2 final prueba solo nodo3

Análisis de las Gráficas y Dashboards de Grafana

1. Dashboard "BD2 — Estado del Clúster":

- Disponibilidad de Targets: En operación normal, los 9 objetivos de scraping reportaron estado HEALTHY continuo.
- Lag de Replicación: Se mantuvo en 0 bytes durante toda la operación normal, demostrando que la réplica síncrona (Nodo 2) se mantuvo en sincronía en tiempo real con el líder.
- Visualización de Roles: La gráfica muestra con precisión el cambio de estado cuando Nodo 1 cae a 0 en patroni_primary y Nodo 2 asciende a 1.0.

2. Dashboard "BD2 — Carga y Fallos":

- Throughput y Errores: Se observa una meseta sostenida de ~101 operaciones/segundo. En el momento exacto de la falla provocada al 50%, la gráfica refleja un "valle" de recuperación de pocos segundos donde se registran los 18 errores y luego retoma su nivel óptimo sobre el nuevo nodo primario.
- Latencia Percentil (p95 y p99): La latencia p95 se mantuvo estable en ~95-124 ms, con un pico transitorio controlado durante el failover de ~250 ms que volvió a estabilizarse inmediatamente tras la conmutación de HAProxy.

- Consumo de Recursos: El uso de CPU en los nodos bajo carga osciló entre el 10% y el 25%, demostrando que la infraestructura virtualizada en Docker sobre Tailscale es altamente eficiente y no sufrió saturación de recursos.

13. RTO y RPO obtenidos (Fase 8)

1. Registrar el momento exacto de la caída de un nodo.

- Fase 3 (Caída de Nodo 1): T = 2026-09-19 02:57:32.028 UTC
- Fase 4 (Caída de Nodo 2): T = 2026-09-19 03:19:02.734 UTC

2. Registrar el momento en que el servicio vuelve a estar disponible.

- Fase 3 (Promoción de Nodo 2 a TL 2): T = 2026-09-19 02:57:35.208 UTC
- Fase 4 (Promoción de Nodo 1 a TL 3): T = 2026-09-19 03:19:15.734 UTC

3. Calcular el RTO.

Fórmula: $RTO = T_{disponible} - T_{caída}$

Escenario de Falla	T_caída	T_disponible	RTO
Fase 3 (Caída de Nodo 1)	02:57:32.028	02:57:35.208	3.18 s
Fase 4 (Caída de Nodo 2)	03:19:02.734	03:19:15.734	13 s

4. Generar operaciones antes y durante una falla.

- Antes de la falla:

Se insertaron transacciones en databugs.bitacora_pruebas:

- id = 7: 'Registro 1 antes de falla' (Servidor 100.67.149.67).
- id = 8: 'Registro 2 antes de falla' (Servidor 100.67.149.67).

- Durante la falla (Carga de estrés k6):

Se inyectaron 101.35 ops/s constantes. De 24,340 operaciones totales, 24,322 fueron exitosas (99.93% de disponibilidad) y solo 18 fallaron (0.07%) durante la desconexión del socket, sin corromper la base de datos.

5. Recuperar el nodo afectado.

Se ejecutó docker compose start patroni:

1. El nodo reactivado reconoció en etcd que ya existía un líder activo con un timeline superior.
2. Patroni invocó automáticamente pg_rewind para rebobinar las páginas modificadas sin reconstruir la base desde cero.
3. El nodo se acopló en modo réplica (streaming) con Lag = 0, y HAProxy lo reincorporó al pool de lectura (puerto 5001).

6. Comparar la información antes y después de la recuperación.

Se consultó la tabla de bitácora en la réplica sobreviviente (Nodo 3) y en el nodo recuperado:

■ 5 - Prueba Select usando Nodo 3

Todos los datos registrados antes de la caída (IDs 7 y 8) permanecen intactos y en estricto orden secuencial junto a las escrituras posteriores (IDs 40, 73 y 74).

7. Determinar si existió pérdida de datos.

- Transacciones confirmadas: Ninguna transacción que recibió confirmación de COMMIT se perdió.
- Las 18 operaciones fallidas en k6: Fueron transacciones que no lograron completar su handshake TCP al momento exacto del corte (uncommitted), retornando un error limpio al cliente en lugar de quedar en un estado inconsistente.

Entonces se puede decir que no existió una pérdida de datos.

8. Estimar el RPO obtenido.

Gracias a la directiva `synchronous_mode: true`, PostgreSQL no confirma una transacción al cliente hasta que el registro del WAL ha sido escrito en el disco del nodo síncrono (Nodo 2). Al caer el líder, la réplica promovida ya contaba con el 100% de los datos confirmados, garantizando cero pérdida de información.

14. Bitácora de pruebas

En la siguiente bitácora se registran todas las actividades realizadas durante las fases del proyecto:

Fecha y hora	Fase	Nodo / componente	Acción realizada	Resultado obtenido	Evidencia	Observaciones
18/09/2026 19:47	Fase 1	Nodo 1, Nodo2 y Nodo3	Se configuraron 3 nodos en laptops independientes conectadas por Tailscale (IPs privadas 100.x.x.x). Cada nodo corre PostgreSQL + Patroni + etcd en Docker. Nodo 1 primario, Nodo 2 réplica síncrona promovible, Nodo 3 réplica asíncrona con tags nofailover: true/nosync: true. Proxy con HAProxy (pools de escritura/lectura vía endpoints de Patroni) + keepalived (VIP con prioridad Nodo1 >	Conectividad confirmada (tailscale ping). Clúster etcd con 3 miembros en consenso. patronictl list: Nodo1 Leader, Nodo2 Sync Standby, Nodo3 Replica con tags correctos. VIP activa y enrutando.	Verificando Tailscale:  Verificando conectividad:  Verificando etcd:  Verificando patroni:  Verificando grafana:  Verificando dashboard HAProxy: 	Se corrigió una condición de carrera en el primer arranque (cualquier nodo podía ganar la elección inicial, incluso Nodo 3) forzando que Nodo 1 inicie primero. Los tags de Patroni no se pueden fijar por variables de entorno sueltas; se resolvió generando la config al arrancar el contenedor (envsubst).

			Nodo2 > Nodo3) en los 3 nodos.		Cargando dataset: ■ 7 - carga de dat...	
18/09/2026 20:47	Fase 2	Nodo 1, Nodo2 y Nodo3	INSERT/UPDATE/DELETE desde Nodo 1, verificados en Nodo 2 y 3. Mismas operaciones repetidas a través del proxy de Nodo 2.	Replicación correcta y consistente en los 3 nodos. Nodo 2 con lag 0 sostenido; Nodo 3 con lag momentáneo que se resuelve en segundos. Escrituras vía proxy de Nodo 2 ejecutadas correctamente sobre el líder real. INSERT directo a Nodo 2 rechazado (read-only transaction), confirmando solo-lectura a nivel de motor.	Inserción de registros desde Nodo 1: ■ 1 - Inserción de... Verificación de registros en Nodo 2 y Nodo 3: ■ 2.1 - Consulta ... ■ 2.2 - consulta d... Actualización de registros en Nodo 1: ■ 3 - Actualizaci... Verificación de cambios en Nodo 2 y Nodo 3: ■ 4.1 - consulta d... ■ 4.2 - consulta u... Eliminación de registros desde Nodo 1: ■ 5 - Eliminación...	El proxy redirige la escritura al líder real sin importar a cuál nodo se conecte el cliente. La asignación de Sync Standby tarda 1-2 ciclos de loop_wait (10s) en estabilizarse, lo cual es un comportamiento esperado.

					Verificación de eliminación en los demás nodos: 6.1 - Consulta ... 6.3 consulta de ... Operaciones de inserción, actualización y eliminación desde Nodo 2: 7 - Insert, Upda... Verificación de cambios en Nodo 1 y Nodo3: 8.1 - Consulta ... 8.2 - consulta d...	
18/09/2026 20:55	Fase 3	Nodo 1, Nodo2 y Nodo3	Insert de control, detener Nodo 1, verificar detección por el proxy, ejecutar lectura/escritura contra la VIP, verificar propagación a Nodo 3, reiniciar Nodo 1.	Nodo 2 promovido automáticamente a líder. HAProxy redirigió las escrituras sin cambios del lado del cliente. Nodo 3 siguió recibiendo cambios. Nodo 1 se	Verificación inicial de nodos: 1 - verificacion... Inserción de información antes de la falla: 2 - Inserciones ...	Nodo 3 nunca fue candidato a líder (nofailover: true funcionó correctamente bajo falla real).

				reincorporó como réplica sin reclamar el liderazgo (sin split-brain). Tiempo de recuperación: 3.18s	Detener de Nodo 1:  3 - Detener ser... Verificación en proxy:  4 - verificación... Ejecución de operaciones de lectura y escritura en Nodo 2:  4 - verificación... Verificación de detección cambios en Nodo 3:  6 - prueba de le... Encender de Nodo 1:  7 - Encender n... Verificación de reintegración y sincronización:  8 - Verificar rei...	
--	--	--	--	---	--	--

18/09/2026 21:14	Fase 4	Nodo 1, Nodo2 y Nodo3	Detener Nodo 2, verificar detección por el proxy, ejecutar lectura/escritura contra Nodo 1, verificar propagación a Nodo 3, reiniciar Nodo 2.	Nodo 1 siguió sirviendo sin interrupción. Nodo 2 se reincorporó como réplica síncrona automáticamente. Tiempo de recuperación: 13s.	Verificación inicial de nodos: ■ 1 - verificación... Detener Nodo 2: ■ 2 - Detener ser... Verificación de proxy: ■ 3 - nodos activ... Ejecución de operaciones de lectura y escritura en Nodo 1: ■ 4 - Pruebas de ... Verificación de cambios desde Nodo 3: ■ 5 - consulta des... Encender Nodo 2: ■ 6 - Iniciar servi... Verificación de reintegración y sincronización:	—
---------------------	--------	--------------------------	---	--	--	---

					 7 - verificacion...	
18/09/2026 21:25	Fase 5	Nodo 1, Nodo2 y Nodo3	Apagar Nodo 1 y Nodo 2, verificar que la escritura deje de estar disponible, ejecutar SELECT contra Nodo 3, reencender un nodo y verificar recuperación de escritura, reencender el restante y verificar sincronización total.	Pool de escritura vacío, escrituras rechazadas correctamente. SELECT contra Nodo 3. Al reencender el primer nodo, escritura recuperada. Al reencender el segundo, sincronización completa de los 3.	Verificación inicial de nodos:  1 - verificación... Apagar Nodo 1:  2 - Apangando ... Apagador Nodo 2:  3 - Apagado N... Verificación de bloqueo en operaciones de escritura:  4 - verificacion...  6 - prueba escri... Ejecución de consultas SELECT desde Nodo 3:  5 - Prueba Sele... Encender Nodo 2:  7 - Iniciando N...	—

					Verificación de la recuperación del servicio de escritura: ■ 8 - insercion co... Encender Nodo 1: ■ 9 - Encendiend...	
18/09/2026 22:00	Fase 6	Nodo1, Nodo2 y Nodo3	Arrancar prueba con K6 (Inserción y lectura) 1. Apagado de nodo2 Monitorizar Arrancar prueba(lectura y escritura) 2. Apagado de nodo1 Monitorizar Apagar nodo1 Apagar nodo2 Arrancar prueba de escritura Monitorizar	Al apagar el nodo 2 los 23,741 operaciones se hicieron con éxito Al apagar el nodo1, 18 inserciones y lecturas fallaron de los 24,340 operaciones y al momento de apagar los dos nos todas las lecturas se hicieron con exito y se hicieron las 11850 lecturas con exito	Arrancar prueba de escritura y lectura ■ 1 - inicio de pr... Detener nodo2 ■ 2 - Deteniendo ... Monitoreo durante la falla ■ 3 - durante la f... Final de la prueba ■ 4 - fin de la pru... Arrancar prueba de escritura y lectura ■ 5 - inicio de pr... Detener nodo1 ■ 6 - Deteniendo ... monitoreo durante la prueba	—

					■ 8 - durante la f... Final de prueba ■ 9 - fin de la pru... Inicio Prueba solo lectura ■ 10 - falla de no... Final prueba solo lectura ■ 11 - fin de prue...	
18/09/2026 22:28	Fase 7	Nodo1, Nodo2 y Nodo3	Inicialización de sistema de monitor repetición de las pruebas de la fase6	Al apagar el nodo1 se observó 18 fallas y 24270 operaciones exitosas Al apagar el nodo2 se observó 20 fallas y 24290 operaciones exitosas en ambos casos se observaron picos de consumo y pequeñas	Inicio de prueba ■ 1.1 - prueba no... ■ 1.2 - prueba no... Apagado de nodo1 ■ 2 - Deteniendo ... monitoreo durante la falla ■ 3.1 - durante la ... ■ 3.2 - durante la ... resultados finales de prueba ■ 4.1 - final de la... ■ 4.2 - final de la...	durante la prueba de solo lectura se observó una estabilidad durante el procesamiento de las operaciones

			interrupciones durante el apagado de los nodos La prueba de solo lectura, se vio una estabilidad en el procesamiento de las operaciones	Inicio de prueba 6.1 - prueba no... 6.2 prueba nod... Apagado de nodo2 7 - Deteniendo ... Monitoreo durante la falla 8.1 - durante la ... 8.2 - durante la ... Final de la prueba 9.1 - fianl de la... 9.2 - final de la... Inicio de prueba de solo lectura (sin nodo1 y nodo2) 10.1 inicio de p... 10.2 inicio de p... Final de prueba de solo lectura 11.1 final prue... 11.2 final prue...	
--	--	--	---	---	--

18/09/2026 20:50	Fase 8	Nodo1 y Nodo2	Al momento de apagar el nodo1/nodo2 se midió el tiempo hasta que el nodo1/nodo2 toma el lugar del que se cayó Al momento de encender nodo1/nodo2 se tomó el tiempo hasta que nodo1/nodo2 tomo su lugar de réplica	RTO nodo1: 3.18 segundos nodo2: 13 segundos RPO = 0 segundos, 0 transacciones, en los tres escenarios.	txt de tiempos de recuperación https://drive.google.com/file/d/1RkaE1YFYqPu_5ZF6bjfWY7_e28aFTILg/view?usp=drive_link	–
---------------------	--------	---------------	---	--	--	---

15. Ventajas y limitaciones

Ventajas

- Failover completamente automático: No requiere intervención humana. Los componentes que deciden, ejecutan, enrutan y operan de forma autónoma y coordinada.
- Pérdida de datos nula en failover principal: La replicación síncrona hacia el segundo nodo garantiza que toda transacción confirmada al cliente existe en al menos dos nodos.
- Transparencia para el cliente: La aplicación se conecta a un puerto estable, el 5000 para escritura y el 5001 para lectura sin conocer ni actualizar la topología del clúster.
- Separación de cargas: El líder queda excluido automáticamente del pool de lectura, dedicando sus recursos a las escrituras mientras las consultas se distribuyen entre las réplicas.
- Degradación controlada: ante la pérdida de dos nodos el sistema no colapsa, sólo renuncia a las escrituras y aún se conserva el acceso a las lecturas.
- Entornos independientes: Los tres nodos operan sobre máquinas físicas distintas, en ubicaciones geográficas distintas y bajo proveedores de Internet distintos.
- Observabilidad: El estado del clúster, el retraso de replicación y el comportamiento bajo carga son observables en tiempo real y quedan registrados históricamente, lo que permite medir en lugar de estimar.

Limitaciones

- El script de vigilancia de keepalived no cede la VIP: El parámetro weight 2 resulta insuficiente frente a una brecha de prioridades de 50 puntos. Aun resolviendo el problema de enrutamiento anterior, la caída de HAProxy en el nodo maestro no provocaría la migración de la dirección virtual
- Un solo nodo acepta escrituras: La capacidad de escritura no escala horizontalmente: añadir nodos aumenta la disponibilidad y la capacidad de lectura, no la de escritura.
- La pérdida de dos nodos elimina las escrituras: Con quórum de 2 sobre 3, la tolerancia a fallos se limita a un nodo para operación completa.
- Tailscale es una dependencia externa: Toda la comunicación entre nodos atraviesa el tailnet, Una falla del servicio de coordinación de Tailscale afectaría al clúster completo.

16. Mejoras futuras

Sustituir la VIP por cadena de conexión multi-host

PostgreSQL admite de forma nativa múltiples servidores en la cadena de conexión: El cliente intenta cada dirección en orden hasta encontrar una que satisfaga el atributo solicitado. Esto proporciona failover del lado del cliente sin depender de una IP virtual, resolviendo de raíz la limitación identificada en Tailscale y eliminando por completo la necesidad de keepalived.

Incorporar respaldos con recuperación a un punto en el tiempo

Herramientas como pgBackRest o Barman añadirían una capa de protección independiente de la replicación, cubriendo el escenario de error lógico o corrupción

Trasladar el monitoreo a un equipo independiente

Eliminaría el punto único de fallo compartido entre observabilidad y nodo de contingencia, y suprimiría el ruido que el generador de carga introduce en las métricas.

Configurar alertas automáticas

Alertmanager podría notificar la caída de un nodo o un retraso de replicación excesivo, en lugar de depender de la observación manual del panel

Incrementar el clúster a cinco nodos

Elevaría el quórum a 3, permitiendo tolerar la caída de dos nodos manteniendo la capacidad de escritura.

17. Conclusiones

La arquitectura cumple su objetivo principal: El sistema mantiene la continuidad del servicio ante la caída de cualquier nodo individual, con recuperación automática y sin intervención humana. Las pruebas confirmaron promoción automática de réplicas, reintegración correcta de nodos recuperados y conservación del acceso de lectura incluso con dos de los tres nodos fuera de servicio.

La separación de responsabilidades resultó determinante: Asignar la decisión a etcd, la ejecución a Patroni y el enrutamiento a HAProxy sin que ninguno invada la competencia del otro evita las decisiones contradictorias que aparecerían si dos componentes intentaran determinar simultáneamente quién es el primario.

La replicación síncrona demostró ser la decisión correcta: Acepta un coste medible en latencia de escritura a cambio de garantizar que ninguna transacción confirmada se pierde en un failover.

El tiempo de recuperación no es un valor único: Una de las conclusiones técnicas más relevantes del trabajo es que existen al menos tres tiempos distintos, con órdenes de magnitud diferentes: la promoción del nuevo líder en segundos, el restablecimiento efectivo del servicio para el cliente en segundos, pero más tardado, y la recuperación de la redundancia completa cuando un nodo debe reconstruirse en minutos

El entorno distribuido real aportó valor formativo por encima de la simulación: Ejecutar los nodos en máquinas físicas separadas, sobre redes domésticas distintas, expuso una clase de problemas que un entorno controlado nunca habría revelado: incompatibilidades de red entre entornos de ejecución, conflictos de puertos con servicios preexistentes del sistema operativo, divergencias de configuración entre integrantes, resolver esos problemas constituyó una parte fundamental del aprendizaje del proyecto.

18. Anexos

Link del repo:

<https://github.com/HerberthReyes07/BD2-proyecto1.git>

Link de los diagramas:

<https://drive.google.com/file/d/1z0jrkJRnqwtnJv2lmyIUGa4h4czVVEQN/view?usp=sharing>